

1 UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
2 SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
3 DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
4 DELL'INFORMAZIONE
5 CORSO DI LAUREA TRIENNALE IN INFORMATICA

6 A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
7 AT UNINA

8 **Relatori**

Megadir. Prof. Guidobaldo Maria
RICCARDELLI
Duca Conte Francesco BARAMBANI
Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier
Carlo ing. SEMENZARA

Candidato

Marco FESTA
N86004979

Sommario

1

2 Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo
3 di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interroga-
4 zione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel
5 contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization*
6 *and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza
7 Artificiale volto a supportare la Pubblica Amministrazione Locale nell'elaborazione e
8 stesura di atti amministrativi.

9 Il problema tecnico principale affrontato in questo lavoro è la trasformazione di do-
10 cumenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine
11 (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Tito-
12 li, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo
13 **Neo4j**, permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi
14 originali.

15 Per abilitare il recupero intelligente e contestuale delle informazioni, è stato implemen-
16 tato un sistema di **Semantic Search**. Questa fase ha richiesto uno studio approfondito e
17 il testing di molteplici **modelli di embedding**, allo scopo di individuare la soluzione più
18 performante per la vettorializzazione del complesso gergo giuridico italiano. Al fine di
19 validare le scelte architetturali, è stato inoltre introdotto e impiegato il database vetto-
20 riale **Qdrant** a scopo puramente sperimentale. Ciò ha permesso di condurre un'analisi
21 comparativa diretta tra le performance di un approccio di ricerca puramente vettoriale e i
22 vantaggi offerti dalla navigazione topologica nel grafo.

23 I risultati di questo lavoro pongono solide basi per l'ottimizzazione del recupero
24 dell'informazione in ambito legale, dimostrando l'efficacia di un **approccio ibrido** che
25 unisce la potenza della ricerca semantica alla precisione strutturale dei database a grafo.

Indice

2	1 Introduzione	1
3	1.1 Definizione del problema	1
4	1.2 Tecnologie utilizzate	2
5	2 Architettura del Sistema	4
6	2.1 Componenti del Sistema	4
7	3 Implementazione Software	6
8	3.1 Estrazione dei Documenti	6
9	3.1.1 Open Data Loader e la Gestione dei Tagged PDF	6
10	3.1.2 Docling e l'Integrazione dei Motori OCR	7
11	3.1.3 Progettazione del Custom Triage Processor	8
12	3.2 Strutturazione Semantica dei Documenti tramite LLM	11
13	3.2.1 Il Modello e la Strategia di Generazione	11
14	3.2.2 Mappatura dello Schema	11
15	3.2.3 Estrazione delle Relazioni Semantiche	12
16	3.3 Importazione dati nei Database	13
17	3.3.1 I Modelli di Dominio	14
18	3.3.2 Disaccoppiamento della Persistenza: Lo Strategy Pattern	14
19	3.3.3 Modalità di Accesso ai Dati: Il Repository Pattern	15
20	3.3.4 Dettagli Implementativi dell'Ingestione su Neo4j	15
21	3.3.5 Dettagli Implementativi dell'Ingestione su Qdrant	16
22	3.4 Creazione degli Embedding	17
23	3.4.1 Astrazione e Gestione dei Modelli di Embedding	17
24	3.4.2 Integrazione con i Database di Memorizzazione	18
25	3.4.3 Orchestrazione del Processo	19
26	Appendice A Codice Sorgente	20
27	A.1 Interfaccia Base	20
28	A.2 Implementazione delle Strategie	20
29	A.2.1 Strategia per Tagged PDF	20
30	A.2.2 Custom Triage Processor	21
31	A.3 Logica di Selezione della Strategia	22

1	A.4 Esecuzione Principale	23
---	-------------------------------------	----

-1-

Introduzione

CONTENTS: 1.1 Definizione del problema. 1.2 Tecnologie utilizzate.

1.1 Definizione del problema

Il lavoro di tesi si inserisce nel contesto delle attività di ricerca e sviluppo svolte presso InfoCube, un'azienda di consulenza informatica specializzata, tra le altre cose, in Intelligenza Artificiale. Nello specifico, il progetto ruota attorno a LUDOVICA (*Language Understanding for Document Optimization and Validation In Civil Administration*), un motore generativo basato sull'IA progettato per la Pubblica Amministrazione Locale. L'obiettivo principale di LUDOVICA è fornire ai funzionari un sistema capace di elaborare richieste in linguaggio naturale, cercare atti pertinenti, e generare template di atti amministrativi pronti all'uso e modificabili.

Il problema tecnico fondamentale affrontato in questo tirocinio consiste nella **trasformazione di documenti legali in un formato strutturato e interrogabile dalle macchine, in modo da poter costruire un solido database a grafo (*Knowledge Graph*)**. Poiché gli atti normativi possiedono una gerarchia fissa e complessa (Titoli, Capi, Articoli, Commi), è cruciale che l'architettura dei dati rifletta questa struttura per permettere al sistema di risalire sempre al documento originale a partire da una singola porzione di testo.

A livello operativo, le attività di tirocinio si sono concentrate su tre task principali:

1. Il **salvataggio dei documenti legali e delle loro gerarchie** sul database a grafo *Neo4j*.
2. L'**implementazione della *Semantic Search***, affiancata al **testing di molteplici modelli di embedding** per valutarne le prestazioni e l'accuratezza delle risposte sul testo legale in lingua italiana.

- 1 3. L'introduzione del database vettoriale **Qdrant**, impiegato unicamente a scopo
2 di testing per poter effettuare un **confronto diretto delle performance** tra un
3 approccio basato sulla sola ricerca vettoriale e quello topologico basato sui grafi.

4 1.2 Tecnologie utilizzate

5 Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei
6 documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di
7 parsing avanzato, modelli di LLM ed embedding e database specializzati:

8 Linguaggi di Programmazione

- 9 • **Python**: Scelto per la vastità e la potenza delle sue librerie, è stato il linguaggio
10 cardine per la costruzione delle pipeline del progetto.

11 Strumenti per l'Estrazione del Testo

- 12 • **Open Data Loader**: Una libreria open-source scelta per la sua capacità di estrarre
13 testo in modo *deterministico* da file PDF, convertendolo direttamente in Markdown.
- 14 • **Docling**: Una libreria open-source di parsing e analisi del layout documentale,
15 specializzato nell'estrazione precisa di testo, tabelle e metadati strutturati a partire
16 da formati complessi come i PDF. Integrato come supporto di Open Data Loader.
- 17 • **RapidOCR**: Modulo impiegato per estrarre testo dai documenti derivanti da scan-
18 sioni visive. Tra i vari motori OCR disponibili, è stato optato l'uso di questo specifico
19 OCR che si è rivelato il compromesso più efficace per bilanciare i tempi di esecuzione
20 e l'elevato consumo di RAM del container.

21 Database e Linguaggi di Interrogazione

- 22 • **Neo4j**: Il database a grafo principale utilizzato per mappare la rigida struttura
23 gerarchica degli atti normativi e i nodi concettuali. Consente di gestire e navigare le
24 relazioni in maniera altamente efficiente rispetto ai database relazionali tradizionali.
- 25 • **Cypher**: Il linguaggio di interrogazione dichiarativo nativo di Neo4j, utilizzato per
26 effettuare le query di inserimento (ingestion) e di ricerca topologica nel grafo.
- 27 • **Qdrant**: Un database vettoriale utilizzato sperimentalmente per condurre test e
28 confronti sulle performance della ricerca semantica (*Cosine Similarity*) rispetto
29 all'approccio basato sui grafi.

1 LLM e Modelli di Embedding

- 2 • **Gemini 2.5-flash:** *Large Language Model* utilizzato tramite API per il compito
3 cruciale della strutturazione sintattica. Riceveva in input il testo in formato Mar-
4 kdown dai PDF e restituiva JSON formattati rigorosamente secondo uno schema
5 predefinito, che rappresentavano la gerarchia dei documenti normativi (Titoli, Capi,
6 Articoli, Commi) e i nodi concettuali.
- 7 • **Modelli di Embedding:** Sono stati testati svariati modelli per ottimizzare la vet-
8 torializzazione del testo legale in italiano. I test più approfonditi hanno coinvolto i se-
9 guenti modelli: `intfloat/multilingual-e5-large`, `google/gemini-embedding-001`
10 e `BAAI/bge-m3`. La valutazione si è basata sulla capacità di recuperare commi
11 pertinenti a partire da query in linguaggio naturale.

12 Deployment e Containerizzazione

- 13 • **Docker:** L'intero ecosistema è stato modularizzato attraverso lo sviluppo di contai-
14 ner isolati e specifici (es. `extract-from-pdf`, `db-ingestion`, ...). Questa scelta
15 ha garantito un'elevata modularità, l'isolamento delle dipendenze software e un
16 rigoroso controllo sulle risorse computazionali, specialmente durante i task più
17 onerosi legati all'*Optical Character Recognition (OCR)* e alla *generazione degli*
18 *Embedding*.

-2-

Architettura del Sistema

3 CONTENTS: 2.1 Componenti del Sistema.

4 In questo capitolo viene presentata l'architettura complessiva del sistema sviluppato,
5 illustrando i componenti principali e il flusso di dati attraverso le diverse fasi del processo.
6 L'obiettivo è fornire una panoramica chiara e dettagliata delle scelte progettuali adottate
7 per affrontare le sfide tecniche del progetto.

8 2.1 Componenti del Sistema

9 I componenti sono presentati in ordine logico, rappresentando il percorso che i dati
10 seguono dal momento in cui vengono acquisiti fino alla fase di interrogazione e utilizzo
11 da parte degli utenti finali.

12 1. **Estrazione dei Documenti:** Descrizione delle tecniche utilizzate per l'estrazione
13 dei testi dai documenti normativi, con particolare attenzione alle strategie e alle
14 librerie impiegate per il parsing e la pulizia dei dati.

15 2. **Strutturazione dei Documenti:** Spiegazione del ruolo dei LLM nella trasfor-
16 mazione del testo estratto in un formato strutturato, con particolare attenzione
17 alla definizione dello schema JSON utilizzato per rappresentare la gerarchia dei
18 documenti normativi.

19 3. **Archiviazione nei Database:** Descrizione delle tecnologie di storage utilizzate,
20 con focus sui due principali database impiegati:

21 3.1 **Database a Grafo (Neo4j):** Illustrazione della struttura del database e del
22 processo di inserimento dei dati, con particolare attenzione alla modellazio-
23 ne delle informazioni e alla rappresentazione delle relazioni gerarchiche dei
24 documenti normativi.

- 1 **3.2 Database Vettoriale (Qdrant):** Descrizione del processo di inserimento dei
 2 dati strutturati nel database
- 3 **4. Implementazione della Ricerca Semantica:** Spiegazione dell'approccio adottato
 4 per la ricerca semantica, compresa la scelta dei modelli di embedding e le definizioni
 5 delle metriche di valutazione.
- 6 **5. Benchmarking e Confronto tra Database:** Processo di benchmark condotto per
 7 valutare le prestazioni di ricerca, analizzando le differenze e le performance tra
 8 l'approccio basato su database a grafo e quello basato su database vettoriale.
- 9 La figura 2.1 fornisce un diagramma riassuntivo dell'architettura del sistema, evidenziando
 10 i componenti principali e il flusso di dati tra di essi.

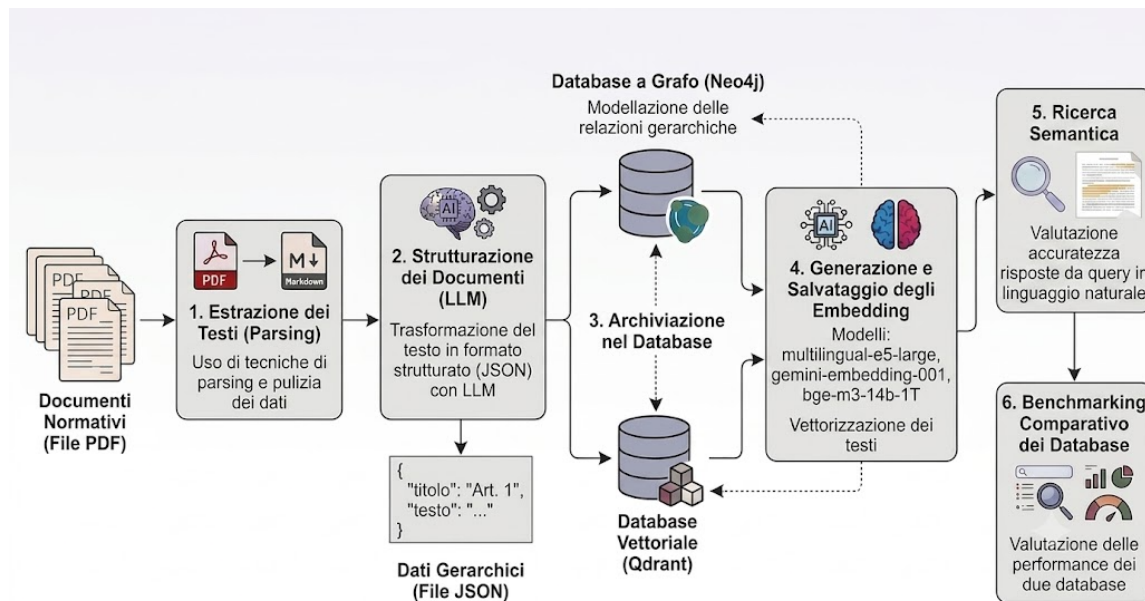


Figura 2.1: Architettura del sistema

–3–

Implementazione Software

CONTENTS: 3.1 Estrazione dei Documenti. 3.2 Strutturazione Semantica dei Documenti tramite LLM. 3.3 Importazione dati nei Database. 3.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione semantica operata dai Large Language Models, fino ad arrivare alle logiche di ingestione e interrogazione sui database

3.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model (Gemini), demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

3.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento consente di convertire localmente i file PDF in formati pronti per l'alimentazione degli LLM, specificamente Markdown e JSON, garantendo un'accurata ricostruzione dell'ordine di lettura e un'efficace estrazione delle tabelle e dei relativi bounding box associati agli elementi grafici. Il vantaggio di questa soluzione risiede nella sua natura puramente deterministica: a parità di input viene

1 garantito lo stesso identico output, azzerando le variabilità tipiche dei modelli generativi
2 in fase di pura lettura del testo.

3 Una delle principali problematiche riscontrate nell'estrazione da documenti legali
4 riguarda la presenza di a capo spuri all'interno del file PDF originale, i quali, se non gestiti,
5 introducono doppi ritorni a capo (`\n\n`) nel testo Markdown, frammentando artificialmen-
6 te i periodi e inficiando la qualità della successiva fase di chunking. Per risolvere questa
7 criticità, all'interno dello script di estrazione è stato configurato il parametro specifico:
8 `use_struct_tree=True`.

9 Quando questo parametro è abilitato, la libreria verifica se il file analizzato appartiene
10 alla categoria dei *Tagged PDF*. A differenza dei PDF standard, che si limitano a definire
11 il layout visivo e geometrico della pagina, i Tagged PDF incorporano un livello logico
12 aggiuntivo e invisibile all'utente: il *Logical Structure Tree* (Albero della Struttura Logica).
13 Questo albero gerarchico è composto da elementi strutturali (*Structural Element Nodes*)
14 paragonabili ai tag del linguaggio HTML (quali `<H1>`, `<P>`, `<Table>`, `<L>` per le liste).
15 Ogni porzione di testo visibile sulla pagina è mappata in modo univoco all'interno di uno
16 di questi nodi, specificandone esplicitamente il ruolo semantico e l'esatto ordine logico
17 di lettura, indipendentemente dalla disposizione grafica dei blocchi di testo nello spazio
18 bidimensionale.

19 L'abilitazione di `use_struct_tree=True` permette quindi a Open Data Loader di
20 ignorare completamente le euristiche visive o geometriche di estrazione — che spesso falli-
21 scono o introducono rumore — e di navigare direttamente lo *Structure Tree* del documento.
22 Questa tecnica consente di discriminare con precisione assoluta gli a capo puramente
23 stilistici (dovuti ad esempio dal raggiungimento del margine della pagina) dalle effettive
24 interruzioni di paragrafo, restituendo un testo continuo, pulito e semanticamente fedele
25 all'originale.

26 Nel contesto specifico di questo lavoro di tesi, focalizzato sui documenti normativi
27 e sugli atti amministrativi della Pubblica Amministrazione Locale, l'approccio basato
28 sui Tagged PDF si è rivelato straordinariamente efficace per ragioni sia pratiche che
29 legali. Le istituzioni pubbliche sono infatti vincolate da stringenti obblighi normativi in
30 materia di accessibilità digitale. Tali norme impongono che ogni documento pubblicato
31 sui portali istituzionali sia pienamente fruibile anche da utenti con disabilità visive o
32 motorie attraverso l'uso di tecnologie assistive, come gli screen reader.

33 La conformità a questi standard garantisce che il *Logical Structure Tree* del PDF non
34 solo sia presente, ma risulti strutturato accuratamente: i testi seguono una gerarchia
35 rigida, le tabelle possiedono intestazioni chiare per righe e colonne, e l'ordine logico di
36 lettura è validato direttamente alla fonte. Di conseguenza, quello che per l'ente pubblico
37 rappresenta un adempimento di legge obbligatorio, si traduce in una vera e propria "ancora
38 di salvezza".

39 3.1.2 Docling e l'Integrazione dei Motori OCR

40 L'approccio standard basato su Open Data Loader mostra evidenti limiti in presenza di
41 documenti non strutturati, privi di testo selezionabile o derivanti da scansioni visive,

in quanto la libreria non integra nativamente un modulo OCR. Per sopperire a questa mancanza e gestire layout documentali complessi, è stata introdotta nel sistema la libreria *Docling*, un motore avanzato di parsing e analisi del layout specializzato nell'estrazione di metadati, tabelle e testo da formati complessi, integrato nella pipeline come supporto a Open Data Loader.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione (*extract-from-pdf*) operava infatti con un vincolo stringente di memoria RAM pari a un massimo di 4 GB, per questioni di risorse hardware. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memoria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina e si è proceduto a un'attività di comparazione su tre diversi motori OCR supportati da Docling:

1. **EasyOCR:** Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM superava costantemente la soglia critica dei 4 GB, rendendone impossibile l'utilizzo nell'ambiente di produzione containerizzato.
2. **Tesseract OCR:** Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo all'interno del file Markdown unicamente i link di riferimento alle immagini scansionate.
3. **RapidOCR:** Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ingegneristico ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container, che si è mantenuto costantemente sotto la soglia di guardia. A fronte di un tempo di computazione non trascurabile — quantificabile in circa 18-20 secondi per singola pagina scansionata — questo motore assicura la totale stabilità infrastrutturale dell'intera pipeline.

3.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

Tuttavia, l'applicazione globale con l'uso del parametro `use_struct_tree=True` introduce un bug logico nel triage di default: i documenti non espressamente *tagged* vengono forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure l'attivazione dell'OCR dove necessario. Per risolvere questa problematica è stato progettato e implementato un **Custom Triage Processor**. Il componente agisce secondo la seguente logica di controllo:

- **Fase 1: Verifica preliminare della struttura.** Non appena il file PDF viene acquisito, il sistema ne analizza i metadati per verificare se è codificato come *Tagged PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- **Fase 2: Frazionamento del documento e stima della complessità.** Se il documento risulta privo di struttura (*Non-Tagged*), viene diviso in singole pagine indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.
- **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi precedente, ogni singola pagina subisce un processo di smistamento intelligente per essere indirizzata allo strumento di estrazione più adeguato:
 - Se la pagina presenta un layout lineare e un'elevata densità di testo nativo, viene classificata come *Semplice* e affidata al processo di Open Data Loader.
 - Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma il testo rimane comunque selezionabile, viene classificata come *Complessa* e processata tramite *Docling Standard* (senza abilitare l'OCR).
 - Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come *Scansione* e inviata al container specializzato di *Docling*, dove interviene il motore *RapidOCR* per il riconoscimento ottico dei caratteri.

L'intero processo appena descritto è schematizzato nella Figura 3.1.

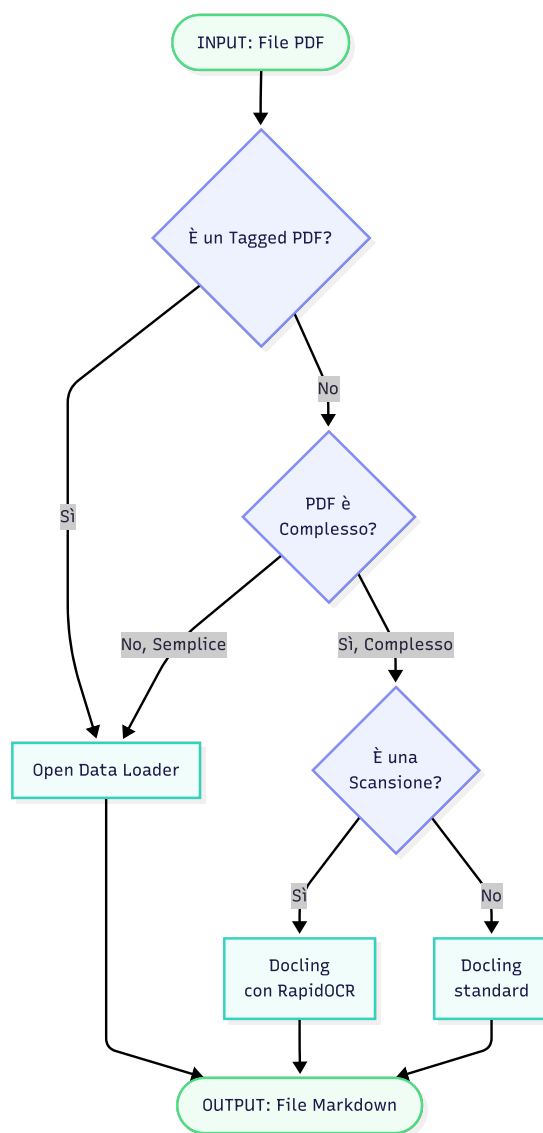


Figura 3.1: Flowchart del Custom Triage Processor

1 Grazie a questa architettura a triage personalizzata, il sistema ha dimostrato una
2 robustezza assoluta durante la fase di testing. Ad esempio, analizzando un batch di 11
3 documenti eterogenei estratti dalla Provincia di Taranto — composto nello specifico da
4 6 documenti nativamente *tagged*, 3 documenti *non-tagged* a testo nativo e 2 documenti
5 interamente scansionati — il sistema ha completato l'intero processo di estrazione in
6 1013.33 secondi (circa 17 minuti), senza registrare alcuna perdita di dati o fallimenti
7 catastrofici dovuti alla memoria.

8 NOTE: Aggiornare le statistiche con i risultati più recenti ottenuti durante la fase di
9 testing finale, includendo il numero totale di pagine elaborate, la distribuzione tra pagine
10 semplici, complesse e scansionate, e il tempo medio di elaborazione per ciascuna categoria.
11 (In teoria saranno 110 file PDF)

1 L'implementazione completa in linguaggio Python del *Custom Triage Processor* è
2 consultabile nell'Appendice A di questo elaborato.

3 **3.2 Strutturazione Semantica dei Documenti tramite** 4 **LLM**

5 Una volta ultimata la fase di estrazione deterministica del testo grezzo, illustrata nella
6 sezione precedente, i documenti normativi si presentano sotto forma di file testuali piatti
7 in formato Markdown. Tali file, pur contenendo l'intero testo originale, sono del tutto
8 privi di metadati strutturali e di una formattazione semantica chiara. Poiché l'architettura
9 di sistema prevede l'ingestione di questi documenti all'interno di un database a grafo
10 (Neo4j) e in un database vettoriale (Qdrant), risulta indispensabile trasformare il testo non
11 strutturato in una gerarchia dati rigorosa. L'archiviazione del solo testo grezzo, infatti,
12 risulterebbe del tutto inadeguata e priva di utilità pratica. Una strutturazione accurata
13 rappresenta il prerequisito fondamentale per abilitare qualsiasi operazione avanzata di
14 *information retrieval*, garantendo che i dati possano essere interrogati in modo efficiente e
15 contestuale.

16 **3.2.1 Il Modello e la Strategia di Generazione**

17 Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large
18 Language Model Gemini 2.5-flash. A livello di architettura software, l'integrazione
19 del modello è stata realizzata mediante il modulo *JsonStrategy*, una classe scritta in
20 Python che implementa il pattern comportamentale *Strategy* interfacciandosi con le API
21 di Google GenAI.

22 Al fine di garantire che l'output del modello sia strettamente confinato a una struttura
23 dati ben definita, la strategia sfrutta la configurazione nativa delle API di Gemini, forzando
24 il parametro *response_mime_type* al valore "application/json". Questo vincolo
25 formale assicura che il modello non restituisca testo discorsivo, introduzioni o blocchi di
26 codice Markdown, ma esclusivamente un oggetto JSON puro, requisito fondamentale per
27 la successiva automazione delle pipeline di inserimento a database.

28 **3.2.2 Mappatura dello Schema**

29 L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt*
30 ingegnerizzato ad hoc. Il modello viene calato nel ruolo di un esperto analista giuridico,
31 con la direttiva assoluta di non riassumere o alterare in alcun modo il significato giuridico
32 del testo originale. Il compito esclusivo del LLM è quello di sanare i difetti di impagina-
33 zione (come i ritorni a capo spuri derivanti dall'estrazione OCR) e mappare il contenuto
34 all'interno del seguente schema logico:

- 1 • **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto.
2 Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome
3 del file originale.
- 4 • **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tut-
5 te le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi
6 (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme
7 (conclusions) e gli eventuali documenti annessi (attachments).
- 8 • **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di struttura-
9 zione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa
10 e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

11 Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono
12 state imposte al LLM regole tassative di parsing:

- 13 • **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente
14 espressi in numeri romani o in forma testuale (es. "TITOLO I", "CAPO IV") devono
15 essere categoricamente convertiti in numeri interi arabi (1, 4, ecc.).
- 16 • **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono
17 sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le
18 interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
- 19 • **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è
20 programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello
21 gerarchico (come un Capo o un Titolo) non trova riscontro nel testo originale, la
22 proprietà viene semplicemente omessa.

23 L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e
24 strutturati gerarchicamente, pronti per essere archiviati e per affrontare il successivo step:
25 l'estrazione delle relazioni semantiche tra le entità legali.

26 3.2.3 Estrazione delle Relazioni Semantiche

27 La sola scomposizione del testo normativo in una struttura gerarchica ad albero (Titoli,
28 Capi, Articoli, Commi) rappresenta una condizione necessaria ma non sufficiente per
29 la costruzione di un vero e proprio *Knowledge Graph*. Per sfruttare appieno la potenza
30 topologica di un database a grafo come Neo4j, è indispensabile individuare e mappare
31 esplicitamente i collegamenti e le dipendenze logiche che intercorrono tra le diverse
32 disposizioni giuridiche all'interno dello stesso atto. A questo scopo, la pipeline prevede
33 un secondo step logico interamente dedicato all'estrazione delle relazioni semantiche.

34 Dal punto di vista implementativo, questo processo si avvale nuovamente dello *Strat-*
35 *egy Pattern*. Come orchestrato dallo script principale `main.py`, una volta conclusa la

1 generazione dei file JSON strutturati, il componente `FileProcessor` adatta il suo comportamento tramite il metodo `set_strategy()`, passando dalla strategia di scomposizione gerarchica a quella di analisi delle relazioni (`RelationshipStrategy`).

4 In questa fase, l'input non è più il testo grezzo, ma i file JSON prodotti nello step precedente. Questi file agiscono come una base di conoscenza consolidata e pulita. Ciascun documento viene quindi analizzato dal LLM Gemini 2.5-flash, guidato da un *System Prompt* specializzato (`gemini-prompt-relationships.md`). Tale prompt è stato progettato per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento formale tra le parti del testo. Le relazioni estratte sono:

- 10 • **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- 11 • **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- 12 • **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- 13 • **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- 14 • **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

15 L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna relazione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste informazioni sono esportate in un nuovo set di file JSON.

18 Questo approccio a due stadi—prima l'estrazione della struttura e poi l'analisi delle relazioni—permette di separare le responsabilità e ridurre il carico cognitivo del modello a ogni ciclo. In questo modo, si minimizza il rischio di errori e si produce un dataset pulito e affidabile, pronto per essere caricato nel grafo.

22 3.3 Importazione dati nei Database

23 Una volta completata l'estrazione e la strutturazione dei documenti legali in formato JSON, si apre la fase di persistenza e indicizzazione dei dati all'interno dei database. Nel contesto di questo progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno di due distinte tipologie di database: Neo4j, un database a grafo, e Qdrant, un database vettoriale.

28 L'uso di due database consente di sperimentare l'uso di entrambi i paradigmi di memorizzazione sullo stesso dominio normativo, al fine di valutarne i rispettivi punti di forza, analizzare le performance e far emergere differenze qualitative e quantitative nelle fasi di interrogazione e navigazione dei testi legali.

32 Per garantire la coesistenza flessibile di queste due tecnologie e favorire un ambiente di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione combinata di due noti design pattern software: lo *Strategy Pattern* e il *Repository Pattern*.

1 3.3.1 I Modelli di Dominio

2 Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati
3 estratti vengano rappresentati in memoria. All'interno della cartella `models`, il sistema
4 definisce le entità fondamentali del dominio normativo utilizzando il paradigma ad oggetti.
5 Queste classi operano essenzialmente come *Models*: strutture dati pure, prive di logica di
6 persistenza diretta o dipendenze verso i DBMS, il cui unico scopo è incapsulare i dati e
7 preservare i vincoli di integrità del dominio.

8 L'architettura dei modelli rispecchia fedelmente la scomposizione gerarchica del testo
9 legale:

- 10 • **Entity**: Rappresenta l'ente pubblico emittente (ad esempio, un Ministero o un
11 Comune), memorizzando i metadati identificativi dell'istituzione.
- 12 • **LegalAct**: Modella l'atto normativo nella sua interezza. Al suo interno, oltre ai meta-
13 dati generali del documento, espone il metodo di fabbrica `from_json_file(file_path)`,
14 delegato al parsing del file JSON e alla deserializzazione ricorsiva dei suoi elementi
15 strutturali.
- 16 • **StructuralElement**: Una classe base polimorfica specializzata nelle sottoclassi
17 `Title`, `Chapter` e `Article`. Permette di mappare i livelli intermedi di aggregazione
18 del testo, mantenendo traccia del numero d'ordine e del titolo della sezione.
- 19 • **Paragraph**: Rappresenta l'unità atomica informativa e testuale, ovvero il singolo
20 comma o blocco di testo non ulteriormente scindibile, sul quale verranno poi calcolati
21 gli embedding.

22 3.3.2 Disaccoppiamento della Persistenza: Lo Strategy Pattern

23 L'esigenza di effettuare una doppia ingestion prima su Neo4j e poi su Qdrant senza dover
24 duplicare la logica di orchestrazione o modificare il flusso principale del programma ha
25 reso naturale l'adozione del design pattern *Strategy*. Questo pattern comportamentale
26 definisce una famiglia di algoritmi, ne incapsula ciascuno all'interno di una classe e li
27 rende tra loro intercambiabili, consentendo all'algoritmo di variare indipendentemente
28 dai client che lo utilizzano.

29 Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe
30 `DBImporterStrategy`. Come mostrato nel listato del codice sorgente, essa definisce tre
31 metodi fondamentali che ogni classe concreta deve implementare:

- 32 • `connect()`: per la gestione e la validazione della connessione con l'istanza specifica
33 del database;
- 34 • `import_legal_act(entity, legal_act)`: per l'esecuzione materiale della logi-
35 ca di Ingestion dell'atto normativo e dei metadati dell'ente emittente;
- 36 • `close()`: per la chiusura sicura delle sessioni.

1 Nello script principale `main.py`, il ciclo di esecuzione batch istanzia dinamicamente
2 l'oggetto concreto (`Neo4jImporter` o `QdrantImporter`) a seconda della configurazione
3 desiderata. La funzione `process_documents` riceve l'istanza polimorfica tramite la *De-*
4 *pendency Injection* ed esegue il parsing e l'ingestion dei file JSON senza curarsi di quale sia
5 il database di destinazione. Questa architettura estendibile riduce a zero l'accoppiamento
6 e semplifica future implementazioni con ulteriori database.

7 3.3.3 Modalità di Accesso ai Dati: Il Repository Pattern

8 Se lo *Strategy Pattern* isola i database nella loro interezza rispetto all'orchestrazione
9 principale, il *Repository Pattern* interviene specificamente all'interno dell'architettura di
10 persistenza dei database per mediare tra lo strato dei modelli di dominio e lo strato di
11 accesso ai dati fisici. L'obiettivo principale del Repository Pattern è centralizzare la logica
12 di costruzione ed esecuzione delle query, offrendo al client un'interfaccia astratta per poter
13 salvare i dati nei database. In questo modo, l'importer concreto delega l'interazione fisica
14 con il database ai singoli repository, i quali ricevono l'oggetto di dominio da persistere,
15 nascondendo completamente i dettagli delle implementazioni delle query.

16 3.3.4 Dettagli Implementativi dell'Ingestione su Neo4j

17 Nel contesto di Neo4j, un atto legale non può essere salvato tramite un'unica operazione
18 atomica standard, a causa delle profonde interconnessioni tra i nodi della gerarchia.
19 Aniché concentrare la scrittura di stringhe Cypher complesse all'interno della classe
20 `Neo4jImporter`, la responsabilità è stata distribuita su una serie di repository specializzati,
21 ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- 22 • `EntityRepository`: Gestisce l'inserimento e l'indicizzazione dei nodi relativi agli
23 enti istituzionali.
- 24 • `LegalActRepository`: Inserisce il nodo radice dell'atto e lo ancora all'ente emit-
25 tente.
- 26 • `StructuralElementRepository`: Incapsula la logica Cypher necessaria a creare i
27 nodi intermedi del grafo e a tessere gli archi gerarchici verso i rispettivi nodi padri.
- 28 • `ParagraphRepository`: Si occupa esclusivamente della persistenza dei nodi foglia
29 contenenti il testo dei commi.

30 Per quanto riguarda il salvataggio delle relazioni trasversali (ad esempio, le citazioni
31 tra diversi atti normativi), queste informazioni vengono lette da file JSON esterni dedicati.

Listing 3.1 Esempio di relazione CITES estratta in formato JSON

```
32 1: {  
33 2:   "sourceId": "ente-provincia-taranto__regolamento-iscrizione-  
34       elenco-speciale-associazioni-e-organismi-senza-scopo-di-  
35       lucro__article-3__paragraph-1",
```

```

1  3:  "relationshipType": "CITES",
2  4:  "targetId": "ente-provincia-taranto__regolamento-iscrizione-
3      elenco-speciale-associazioni-e-organismi-senza-scopo-di-
4      lucro__article-2"
5  5: }

```

Il metodo `import_relationships` esegue di fatto una seconda passata sul grafo e per ogni collegamento estratto dal JSON, l'algoritmo esegue una query Cypher dedicata procedendo alla creazione dell'arco semantico. Inoltre sfruttando l'indicizzazione definita sulla proprietà `custom_id`, il motore di Neo4j individua i nodi con tempi di ricerca estremamente ridotti.

3.3.5 Dettagli Implementativi dell'Ingestione su Qdrant

La strategia implementata nella classe `QdrantImporter` risponde a logiche radicalmente differenti rispetto a quelle dei database a grafo, dettate dai vincoli operativi dei database vettoriali e dalle necessità algoritmiche della ricerca semantica. Qdrant, infatti, non supporta nativamente strutture gerarchiche ramificate, ma organizza le informazioni in collezioni di vettori piatti denominati *Points*.

Il fulcro del processo di ingestione in Qdrant risiede nell'operazione di appiattimento (*flattening*) della struttura ad albero dell'atto normativo, la quale è stata configurata secondo tre fasi logiche:

- **Identificazione dell'unità fondamentale:** L'unità minima di indicizzazione viene individuata nel modello Paragraph (il singolo comma). Questa porzione di testo rappresenta il compromesso ottimale per bilanciare la granularità informativa e la focalizzazione semantica del modello di embedding.
- **Il problema del contesto assoluto:** Memorizzare un comma in modo isolato causerebbe una perdita critica di contesto, rendendo impossibile determinare a quale articolo o sezione appartenga il testo recuperato.
- **Risoluzione tramite Tree-Climbing:** Per preservare l'integrità strutturale, l'algoritmo esegue una procedura di risalita in memoria per ciascun paragrafo. Partendo dall'elemento foglia (il comma), risale ricorsivamente la catena dei nodi padri, estraendo il numero e il titolo di ogni livello gerarchico attraversato (Articolo, Capo, Titolo).

Tutte le informazioni così raccolte vengono infine aggregate e inserite all'interno del dizionario di metadati del punto vettoriale, denominato *Payload*

Listing 3.2 Esempio reale di un Point vettoriale su Qdrant con Payload

```

35  1: {
36  2:  "id": "00020dff-edff-51b8-b2fb-a953ed7e5c0e",
37  3:  "payload": {

```

```

1   4:      "p_id": "ente-provincia-taranto__regolamento-area-delle-
2           posizioni-organizzative-e-delle-alte-
3           professionalita__chapter-2__article-14__paragraph-1",
4   5:      "text": "Il presente Regolamento entrera in vigore ad
5           avvenuta pubblicazione del medesimo all'Albo Pretorio on-
6           line nonche sul sito web ufficiale dell'Ente.",
7   6:      "paragraph_num": "1",
8   7:      "act_name": "Regolamento Area delle Posizioni Organizzative
9           e delle Alte Professionalita.json",
10  8:      "article_num": "14",
11  9:      "article_name": "Entrata in vigore",
12 10:      "chapter_num": "2",
13 11:      "chapter_name": "Area delle Alte Professionalita"
14 12: },
15 13:      "vector": [/* Rappresentazione numerica dell'embedding
16           testuale */]
17 14: }

```

3.4 Creazione degli Embedding

L'operazione di vettorizzazione, o creazione degli *embedding*, rappresenta un passaggio cruciale per abilitare le funzionalità di ricerca semantica sui documenti legali all'interno del sistema.

3.4.1 Astrazione e Gestione dei Modelli di Embedding

Per garantire la massima flessibilità nell'uso di modelli eterogenei, è stata definita un'interfaccia astratta base denominata `EmbeddingModel`. Da essa derivano le implementazioni specifiche per le diverse tipologie di erogazione dei modelli:

- **Modelli Locali (`LocalEmbeddingModel`):** consentono l'esecuzione di modelli open-source direttamente in locale sfruttando la libreria `SentenceTransformer`. I modelli vengono salvati in una cache locale per ottimizzare i caricamenti successivi.
- **Modelli Cloud HuggingFace (`HuggingFaceCloudModel`):** si interfacciano con la *HuggingFace Inference API*. Per garantire la resilienza del sistema, la classe implementa un meccanismo di tolleranza ai guasti basato su tentativi multipli in caso di errore di rete o di superamento dei limiti di utilizzo, con attese esponenziali tra i tentativi.
- **Modelli Cloud Gemini (`GeminiCloudModel`):** comunicano tramite l'endpoint REST `embedContent` di Google. Anche qui è stato implementato il meccanismo di retry e, inoltre, per ottimizzare ulteriormente il throughput, viene utilizzata una strategia di rotazione circolare su un pool di API keys fornite in configurazione.

In particolare, sono stati integrati quattro specifici modelli di embedding per verificare le differenze prestazionali e di accuratezza in fase di retrieval:

- 1 • **google/embeddinggemma-300m**: Si tratta dell'unico modello della suite eseguito
2 interamente in locale sfruttando le risorse hardware della macchina tramite la
3 libreria SentenceTransformer. Produce vettori con una dimensionalità di 768.
- 4 • **intfloat/multilingual-e5-large**: Modello ad alte prestazioni fruibile in cloud
5 tramite le *HuggingFace Inference API*. Caratterizzato da una dimensionalità di 1024
6 vettori, questo modello richiede obbligatoriamente un pre-processing del testo
7 tramite prefissi asimmetrici per ottimizzare le prestazioni di recupero: nello specifico,
8 viene anteposto il token "passage: " per i testi da memorizzare e il token "query:
9 " per le stringhe di ricerca.
- 10 • **BAAI/bge-m3**: Modello cloud multilingua integrato anch'esso tramite l'infrastrut-
11 tura di *HuggingFace*. Genera vettori a 1024 dimensioni ed è particolarmente rino-
12 mato per la sua versatilità nel mappare relazioni semantiche complesse in contesti
13 cross-lingua e multi-lingua.
- 14 • **gemini/gemini-embedding-001**: Modello cloud proprietario di Google accessi-
15 bile mediante l'endpoint REST `embedContent`. Configurato con una dimensionalità
16 di 768 vettori, è nativamente ottimizzato per task di *text retrieval* su larga scala.

17 3.4.2 Integrazione con i Database di Memorizzazione

18 La distribuzione dei vettori generati ai diversi database di memorizzazione viene gestita,
19 anche qui, attraverso lo *Strategy Pattern*, che isola le logiche di inserimento dei dati per
20 ciascun database.

21 Un aspetto cruciale condiviso da entrambe le strategie è l'**arricchimento contestuale**
22 **del testo**. Al fine di massimizzare l'accuratezza della ricerca semantica, il sistema non si
23 limita a generare l'embedding del solo testo grezzo del comma. Al contrario, il testo viene
24 dinamicamente concatenato con i suoi metadati gerarchici e descrittivi (come il nome
25 dell'atto normativo, il numero e il titolo del capitolo, e il numero e la rubrica dell'articolo).

26 La Strategia Neo4j

27 La `Neo4jEmbeddingStrategy` gestisce l'estrazione, la vettorizzazione e il salvataggio
28 degli embedding nel database a grafo. Il processo si articola in tre fasi principali:

- 29 1. **Estrazione e arricchimento contestuale**: Il sistema esegue una query Cypher
30 (`FETCH_QUERY`) che risale la gerarchia del documento a partire dal testo del nodo
31 `:Paragraph`. La query estrae i metadati dei nodi padre (Atto, Titolo, Capo, Arti-
32 colo) e li concatena al contenuto del paragrafo, generando una stringa arricchita e
33 contestualizzata.
- 34 2. **Generazione degli embedding**: Il testo completo viene inviato ai modelli di
35 *embedding*. I modelli processano e traducono il testo in vettori numerici, catturando

1 così sia il significato letterale del testo sia le dipendenze semantiche legate alla sua
2 esatta collocazione nella legge.

3 **3. Salvataggio nel database:** Infine, i vettori generati vengono salvati nel grafo. Vie-
4 ne creato un nuovo nodo dedicato assegnandogli due *label*: una generica (:*Embedding*)
5 e una dinamica, specifica per il modello utilizzato (es. :*Embedding_embeddinggemma_300m*).
6 Questo nodo memorizza il vettore generato (*vector*) e i dettagli del modello. Il
7 processo si conclude creando una relazione [*:HAS_EMBEDDING*] che connette il
8 nodo :*Paragraph* originario al suo rispettivo nodo :*Embedding*.

9 **La Strategia Qdrant**

10 La classe *QdrantEmbeddingStrategy* verifica a runtime lo schema della collezione, ini-
11 zializzandone, ove necessario, la configurazione per i *Named Vectors*. Tale approccio
12 consente di associare a un singolo record vettoriale molteplici rappresentazioni prodotte
13 dai modelli differenti, agevolando le prestazioni in fase di retrieval.

14 In linea con l'approccio adottato per Neo4j, il processo implementa un pre-processing
15 di arricchimento semantico. Prima della generazione dell'embedding, vengono estratti
16 i metadati dal *payload* del database per costruire una gerarchia di contesto testuale (es.
17 "*Atto: Codice Civile | Titolo I | Capo II | Art. 4*").

18 **3.4.3 Orchestrazione del Processo**

19 Il flusso di *ingestion* è governato da uno script principale che, sfruttando un *Factory Pattern*,
20 istanzia dinamicamente tutti i modelli di embedding. Successivamente, esegue un loop
21 iterativo: per ogni modello caricato, lancia l'esecuzione su tutte le strategie di database
22 configurate (Neo4j e Qdrant). L'intero ciclo è tracciato minuziosamente tramite un modulo
23 di logging su file e console per monitorare i tempi di generazione e individuare bottleneck
24 prestazionali.

Appendice A

Codice Sorgente

In questa appendice viene riportato il codice sorgente in linguaggio Python relativo allo step di estrazione dei documenti PDF, con particolare focus sull'implementazione del *Custom Triage Processor*. Il sistema è stato progettato adottando lo *Strategy Pattern*, che permette di definire una famiglia di algoritmi, incapsularli e renderli intercambiabili in base alla natura del documento analizzato.

A.1 Interfaccia Base

Il file `base.py` definisce l'interfaccia astratta che tutte le strategie di elaborazione dei PDF devono implementare.

Listing A.1 Interfaccia base per le strategie

```
11 1: class PDFProcessingStrategy(ABC):
12 2:     @abstractmethod
13 3:     def process(self, pdf_file: Path, reader: PdfReader,
14         temp_export_path: Path) -> str:
15 4:         pass
```

A.2 Implementazione delle Strategie

A.2.1 Strategia per Tagged PDF

Il file `tagged_strategy.py` implementa la strategia per i documenti nativamente strutturati.

Listing A.2 Strategia per PDF strutturati

```
21 1: class TaggedPDFStrategy(PDFProcessingStrategy):
22 2:     def process(self, pdf_file: Path, reader: PdfReader,
23         temp_export_path: Path) -> str:
24 3:         final_markdown = ""
25 4:         convert_args = {
26 5:             "input_path": [str(pdf_file)],
27 6:             "output_dir": str(temp_export_path),
```

```

1   7:         "format": ["markdown"],
2   8:         "use_struct_tree": True,
3   9:         "hybrid_mode": "auto"
4  10:     }
5  11:
6  12:     try:
7  13:         opendataloader_pdf.convert(**convert_args)
8  14:         for temp_md_file in temp_export_path.glob("*.md"):
9  15:             with open(temp_md_file, "r", encoding="utf-8")
10 16:                 as f:
11 17:                 final_markdown += f.read()
12 18:                 temp_md_file.unlink()
13 19:     except Exception as e:
14 20:         logger.error(f"Errore durante l'estrazione di {
15 21:             pdf_file.name}: {e}")
16
17 21:     return final_markdown

```

19 A.2.2 Custom Triage Processor

20 Il file `trriage_strategy.py` contiene il core della logica di smistamento intelligente delle
21 singole pagine.

Listing A.3 Implementazione del Custom Triage Processor

```

22 1: class TriagePDFStrategy(PDFProcessingStrategy):
23 2:     STRATEGY_MAP = {
24 3:         PageType.SIMPLE: {"hybrid_mode": "auto"},
25 4:         PageType.COMPLEX: {"hybrid_mode": "full", "hybrid": "
26         docling-fast", "hybrid_url": URL_DOCLING_NO_OCR},
27 5:         PageType.SCAN: {"hybrid_mode": "full", "hybrid": "
28         docling-fast", "hybrid_url": URL_DOCLING_OCR}
29 6:     }
30 7:
31 8:     def process(self, pdf_file: Path, reader: PdfReader,
32 9:         temp_export_path: Path) -> str:
33 10:         final_markdown = ""
34 11:
35 12:         for i, page in enumerate(reader.pages):
36 13:             page_strategy = self._analyze_page_type(page)
37 14:
38 15:             temp_pdf_path = temp_export_path / f"temp_p{i}.pdf"
39 16:             with open(temp_pdf_path, "wb") as f:
40 17:                 writer = PdfWriter()
41 18:                 writer.add_page(page)
42 19:                 writer.write(f)
43 20:
44 21:             convert_args = {
45 22:                 "input_path": [str(temp_pdf_path)],
46 23:                 "output_dir": str(temp_export_path),
47 24:                 "format": ["markdown"],
48 25:                 "use_struct_tree": False

```

```

1 25:         }
2 26:
3 27:         convert_args.update(self.STRATEGY_MAP.get(
4         page_strategy, self.STRATEGY_MAP[PageType.SIMPLE
5         ]))
6 28:
7 29:         try:
8 30:             opendataloader_pdf.convert(**convert_args)
9 31:             for temp_md_file in temp_export_path.glob("*.md"
10          ):
11 32:                 with open(temp_md_file, "r", encoding="utf-8
12          ):
13 33:                     final_markdown += f.read() + "\n\n---\n\n
14          "
15 34:                     temp_md_file.unlink()
16 35:         except Exception as e:
17 36:             logger.error(f"Errore critico pagina {i+1} di {
18          pdf_file.name}: {e}")
19 37:         finally:
20 38:             if temp_pdf_path.exists():
21 39:                 temp_pdf_path.unlink()
22 40:
23 41:         return final_markdown
24 42:
25 43:     def _analyze_page_type(self, page) -> PageType:
26 44:         text = page.extract_text() or ""
27 45:         has_images = len(page.images) > 0 if hasattr(page, '
28          images') else False
29 46:
30 47:         if len(text.strip()) < 50 and has_images:
31 48:             return PageType.SCAN
32 49:         if has_images or len(text.strip()) > 2500:
33 50:             return PageType.COMPLEX
34 51:
35 52:         return PageType.SIMPLE

```

37 A.3 Logica di Selezione della Strategia

38 Il file `utils.py` funge da *Factory* per il pattern architetturale scelto: analizza i metadati
39 del documento per verificare la presenza di un albero strutturale e istanzia dinamicamente
40 la strategia di elaborazione più adeguata.

Listing A.4 Factory per la selezione della strategia

```

41 1: def get_pdf_strategy(pdf_reader) -> PDFProcessingStrategy:
42 2:     if is_pdf_tagged(pdf_reader):
43 3:         return TaggedPDFStrategy()
44 4:     return TriagePDFStrategy()
45 5:
46 6:
47 7:

```

```
1  8: def is_pdf_tagged(reader: PdfReader) -> bool:
2  9:     try:
3 10:         return "/StructTreeRoot" in reader.trailer["/Root"]
4 11:     except Exception:
5 12:         return False
```

7 A.4 Esecuzione Principale

8 Infine, lo script `main.py` coordina il processo, determinando quale strategia applicare.

Listing A.5 Script principale di esecuzione

```
9  1: def main():
10  2:     setup_logging(LOG_DIR)
11  3:     logger = logging.getLogger(__name__)
12  4:
13  5:     for path in [INPUT_PDF_DIR, BASE_MD_PATH, TEMP_EXPORT_PATH]:
14  6:         path.mkdir(parents=True, exist_ok=True)
15  7:
16  8:     for pdf_file in INPUT_PDF_DIR.iterdir():
17  9:         try:
18 10:             pdfReader = PdfReader(pdf_file)
19 11:             pdf_extraction_strategy = get_pdf_strategy(pdfReader
20 12:                                                         )
21 12:
22 13:             markdown = pdf_extraction_strategy.process(pdf_file,
23 14:                                                         pdfReader, TEMP_EXPORT_PATH)
24 14:
25 15:             with open(BASE_MD_PATH / f"{pdf_file.stem}.md", "w",
26 16:                       encoding="utf-8") as f:
27 17:                 f.write(markdown)
28 17:         except Exception as e:
29 18:             logger.error(f"Errore su {pdf_file.name}: {e}")
30 19:
31 20:     if TEMP_EXPORT_PATH.exists():
32 21:         shutil.rmtree(TEMP_EXPORT_PATH)
33 22:
34 23: if __name__ == "__main__":
35 24:     main()
```

1

Elenco delle figure

2	2.1	Architettura del sistema	5
3	3.1	Flowchart del Custom Triage Processor	10

Elenco delle tabelle